

Theory of Automata

CS-3005

Lecture 17

Mahzaib Younas

Lecturer Department of Computer Science

FAST NUCES CFD

Greibach Normal Form

Conversion of a Chomsky normal form grammar to Greibach normal form

Leftmost Derivation

Definition:

- The **leftmost non-terminal** in a working string is the first non-terminal that we encounter when we scan the string from left to right.

Example: In the string **abNbaXYa**, the leftmost non terminal is **N**.

Definition:

- If a word **w** is generated by a CFG such that at each step in the derivation, a production is applied to the leftmost non-terminal in the working string, then this derivation is called a **leftmost derivation**.

Example: Left derivation

$S \rightarrow aSX \mid b$

$X \rightarrow Xb \mid a$

aababa

$S \rightarrow aSX$

$\rightarrow aaSXX$

$\rightarrow aabXX$

$\rightarrow aabXbX$

$\rightarrow aababX$

$\rightarrow aababa$

Left recursion

- A grammar is left-recursive if we can find some non-terminal A which will eventually derive a sentential form with itself as the left-symbol

$$A \rightarrow Aa \mid b$$

Left Recursion Removal

- Consider the grammar $A \rightarrow Aa \mid b$
- Halting condition of top-down parsing depend upon the generation of terminal prefixes to discover dead ends.
- Repeated application of above rule fail to generate a prefix that can terminate the parse.

Removing left recursion

- For each rule of the form
- $A \rightarrow A \alpha_1 \mid A \alpha_2 \mid \dots \mid A \alpha_n \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$
 - A is a left-recursive nonterminal.
 - α is a sequence of nonterminals and terminals.
 - β is a sequence of nonterminals and terminals that does not start with A .
- Replace the A -production by the production:
- $A \rightarrow \beta_1 A' \mid \dots \mid \beta_n A'$
- And create a new nonterminal
- $A' \rightarrow \Lambda \mid \alpha_1 A' \mid \dots \mid \alpha_n A'$

Removing left recursion

- To remove left recursion from A, the A rules are divided into two groups.

Left recursive and others

$$A \rightarrow AU_1 \mid AU_2 \mid AU_3 \mid \dots \mid AU_j$$

$$A \rightarrow V_1 \mid V_2 \mid V_3 \mid \dots \mid V_k$$

Solution:

$$A \rightarrow V_1 \mid V_2 \mid V_3 \mid \dots \mid V_k \mid V_1Z \mid V_2Z \mid \dots \mid V_kZ$$

$$Z \rightarrow u_1Z \mid u_2Z \mid u_3Z \mid \dots \mid u_jZ \mid u_1 \mid u_2 \mid u_3 \mid \dots \mid u_j$$

Example

- $A \rightarrow Aa \mid b$

Solution:

$$A \rightarrow bZ \mid b$$

$$Z \rightarrow aZ \mid a$$

- $A \rightarrow Aa \mid Ab \mid b \mid c$

Solution:

$$A \rightarrow bZ \mid cZ \mid b \mid c$$

$$Z \rightarrow aZ \mid bZ \mid a \mid b$$

Definition

- A CFG is in Greibach normal form if each rule has one of these forms:
 - i.* $S \rightarrow aA_1A_2...A_n$
 - ii.* $A \rightarrow aA_1A_2...A_n$
 - iii.* $A \rightarrow a$

Conversion

- Convert from Chomsky to Greibach in two steps:
 1. From Chomsky to intermediate grammar
 - a. Eliminate direct left recursion
 - b. Use $A \rightarrow uBv$ rules transformations to improve references (explained later)
 2. From intermediate grammar into Greibach

Eliminate direct left recursion

- Before

$$A \rightarrow A\underline{a} \mid \mathbf{b}$$

- After

$$A \rightarrow \mathbf{b}Z \mid \mathbf{b}$$

$$Z \rightarrow \underline{a}Z \mid \underline{a}$$

- Remove the rule with direct left recursion, and create a new one with recursion on the right

Eliminate direct left recursion

- Before

$$A \rightarrow A\underline{a} \mid A\underline{b} \mid \mathbf{b} \mid \mathbf{c}$$

- After

$$A \rightarrow \mathbf{b}\underline{Z} \mid \mathbf{c}\underline{Z} \mid \mathbf{b} \mid \mathbf{c}$$

$$Z \rightarrow \underline{a}Z \mid \underline{b}Z \mid \underline{a} \mid \underline{b}$$

- Remove the rules with direct left recursion, and create new ones with recursion on the right

Substitution Rule

Transform $A \rightarrow uBv$ rules

- Before

$$A \rightarrow uBb$$

$$B \rightarrow w_1 \mid w_2 \mid \dots \mid w_n$$

- After

$$\text{Add } A \rightarrow uw_1b \mid uw_2b \mid \dots \mid uw_nb$$

$$\text{Delete } A \rightarrow uBb$$

Conversion: Step 1

- Goal: construct intermediate grammar in this format

i. $A \rightarrow aw$

ii. $A \rightarrow Bw$

iii. $S \rightarrow \lambda$

where $w \in V^*$ and B comes after A

Conversion: Step 1

- Assign a number to all variables starting with S, which gets 1
- Transform each rule following the order according to given number from lowest to highest
 - Eliminate direct left recursion
 - If RHS of rule starts with variable with lower order, apply $A \rightarrow uBb$ transformation to fix it

Conversion: Step 2

- Goal: construct Greibach grammar out of intermediate grammar from step 1
- Fix $A \rightarrow Bw$ rules into $A \rightarrow aw$ format
 - After step 1, last original variable should have all its rules starting with a terminal
 - Working from bottom to top, fix all original variables using $A \rightarrow uBb$ transformation technique, so all rules become $A \rightarrow aw$
- Fix introduced recursive rules same way

Conversion Example

- Convert the following grammar from Chomsky normal form, into Greibach normal form
 1. $S \rightarrow AB \mid \lambda$
 2. $A \rightarrow AB \mid CB \mid a$
 3. $B \rightarrow AB \mid b$
 4. $C \rightarrow AC \mid c$

Conversion Strategy

- Goal: transform all rules which RHS does not start with a terminal
- Apply two steps conversion
- Work rules in sequence, eliminating direct left recursion, and enforcing variable reference to higher given number
- Fix all original rules, then new ones

Step 1: S rules

- Starting with S since it has a value of 1
- $S \rightarrow AB \mid \lambda$
- S rules comply with two required conditions
 - There is no direct left recursion
 - Referenced rules A and B have a given number higher than 1. A corresponds to 2 and B to 3.

Step 1: A rules

- $A \rightarrow A\underline{B} \mid \mathbf{CB} \mid a$
- Direct left recursive rule $A \rightarrow AB$ needs to be fixed. Other A rules are fine
- Apply direct left recursion transformation

$$A \rightarrow \mathbf{CB}\underline{R_1} \mid a\underline{R_1} \mid \mathbf{CB} \mid a$$

$$R_1 \rightarrow \underline{B}R_1 \mid \underline{B}$$

Step 1: B rules

- $B \rightarrow \underline{A}B \mid b$
- $B \rightarrow AB$ rule needs to be fixed since B corresponds to 3 and A to 2. B rules can only have on their RHS variables with number equal or higher. Use $A \rightarrow uBb$ transformation technique
- $B \rightarrow \underline{CB}R_1B \mid \underline{a}R_1B \mid \underline{CB}B \mid \underline{a}B \mid b$

Step 1: C rules

- $C \rightarrow \underline{\underline{A}}C \mid c$
- $C \rightarrow AC$ rule needs to be fixed since C corresponds to 4 and A to 2. Use same $A \rightarrow uBb$ transformation technique
- $C \rightarrow \underline{\underline{CB}}R_1C \mid \underline{\underline{a}}R_1C \mid \underline{\underline{CB}}C \mid \underline{\underline{a}}C \mid c$
- Now variable references are fine according to given number, but we introduced direct left recursion in two rules...

Step 1: C rules

- $C \rightarrow C\underline{B}\underline{R}_1\underline{C} \mid \mathbf{a}\mathbf{R}_1\mathbf{C} \mid C\underline{B}\underline{C} \mid \mathbf{a}\mathbf{C} \mid \mathbf{c}$
- Eliminate direct left recursion

$$C \rightarrow \mathbf{a}\mathbf{R}_1\mathbf{C}\underline{R}_2 \mid \mathbf{a}\mathbf{C}\underline{R}_2 \mid \mathbf{c}\underline{R}_2 \mid \mathbf{a}\mathbf{R}_1\mathbf{C} \mid \mathbf{a}\mathbf{C} \mid \mathbf{c}$$

$$R_2 \rightarrow \underline{B}\underline{R}_1\underline{C}\underline{R}_2 \mid \underline{B}\underline{C}\underline{R}_2 \mid \underline{B}\underline{R}_1\underline{C} \mid \underline{B}\underline{C}$$

Step 1: Intermediate grammar

- $S \rightarrow AB \mid \lambda$
- $A \rightarrow CBR_1 \mid aR_1 \mid CB \mid a$
- $B \rightarrow CBR_1B \mid aR_1B \mid CBB \mid aB \mid b$
- $C \rightarrow aR_1CR_2 \mid aCR_2 \mid cR_2 \mid aR_1C \mid aC \mid c$
- $R_1 \rightarrow BR_1 \mid B$
- $R_2 \rightarrow BR_1CR_2 \mid BCR_2 \mid BR_1C \mid BC$

Step 2: Fix starting symbol

- Rules S, A, B and C don't have direct left recursion, and RHS variables are of higher number
- All C rules start with terminal symbol
- Proceed to fix rules B, A and S in bottom-up order, so they start with terminal symbol.
- Use $A \rightarrow uBb$ transformation technique

Step 2: Fixing B rules

- $C \rightarrow aR_1CR_2 \mid aCR_2 \mid cR_2 \mid aR_1C \mid aC \mid c$

- Before

$$B \rightarrow \underline{C}BR_1B \mid aR_1B \mid \underline{C}BB \mid aB \mid b$$

- After

$$B \rightarrow aR_1B \mid aB \mid b$$

$$B \rightarrow \underline{aR_1}\underline{CR_2}BR_1B \mid \underline{aCR_2}BR_1B \mid \underline{cR_2}BR_1B \mid \underline{aR_1}\underline{C}BR_1B \mid \underline{aC}BR_1B \mid \underline{c}BR_1B$$

$$B \rightarrow \underline{aR_1}\underline{CR_2}BB \mid \underline{aCR_2}BB \mid \underline{cR_2}BB \mid \underline{aR_1}\underline{C}BB \mid \underline{aC}BB \mid \underline{c}BB$$

Step 2: Fixing A rules

- $C \rightarrow aR_1CR_2 \mid aCR_2 \mid cR_2 \mid aR_1C \mid aC \mid c$

- Before

$$A \rightarrow \underline{C}BR_1 \mid aR_1 \mid \underline{C}B \mid a$$

- After

$$A \rightarrow aR_1 \mid a$$

$$A \rightarrow \underline{aR_1}\underline{C}\underline{R_2}BR_1 \mid \underline{aC}\underline{R_2}BR_1 \mid \underline{cR_2}BR_1 \mid \underline{aR_1}\underline{C}BR_1 \\ \mid \underline{aC}BR_1 \mid \underline{c}BR_1$$

$$A \rightarrow \underline{aR_1}\underline{C}\underline{R_2}B \mid \underline{aC}\underline{R_2}B \mid \underline{cR_2}B \mid \underline{aR_1}\underline{C}B \mid \underline{aC}B \mid \\ \underline{c}B$$

Step 2: Fixing S rules

- Before

$$S \rightarrow \underline{A}\mathbf{B} \mid \lambda$$

- After

$$S \rightarrow \lambda$$

$$S \rightarrow \underline{aR_1}\mathbf{B} \mid \underline{a}\mathbf{B}$$

$$S \rightarrow \underline{aR_1}\underline{CR_2}\underline{BR_1}\mathbf{B} \mid \underline{aCR_2}\underline{BR_1}\mathbf{B} \mid \underline{cR_2}\underline{BR_1}\mathbf{B} \mid \underline{aR_1}\underline{CBR_1}\mathbf{B} \mid \underline{aCBR_1}\mathbf{B} \mid \underline{cBR_1}\mathbf{B}$$

$$S \rightarrow \underline{aR_1}\underline{CR_2}\underline{BB} \mid \underline{aCR_2}\underline{BB} \mid \underline{cR_2}\underline{BB} \mid \underline{aR_1}\underline{CBB} \mid \underline{aCBB} \mid \underline{cBB}$$

Step 2: Complete conversion

- All original rules S, A, B and C are fully converted now
- New recursive rules need to be converted next

$$R_1 \rightarrow BR_1 \mid B$$

$$R_2 \rightarrow BR_1CR_2 \mid BCR_2 \mid BR_1C \mid BC$$

- Use same $A \rightarrow uBb$ transformation technique replacing starting variable B

Conclusions

- After conversion, since B has 15 rules, and R_1 references B twice, R_1 ends with 30 rules
- Similar for R_2 which references B four times. Therefore, R_2 ends with 60 rules
- All rules start with a terminal symbol (with the exception of $S \rightarrow \lambda$)
- Parsing algorithms top-down or bottom-up would complete on a grammar converted to Greibach normal form

Comparison of Normal forms

$$S \rightarrow Sa\mathbf{B} \mid aB$$

$$B \rightarrow bB \mid \lambda$$

- Adding a non-recursive start symbol S' and removing λ and chain rules yields

$$S' \rightarrow Sa\mathbf{B} \mid Sa \mid aB \mid a$$

$$S \rightarrow Sa\mathbf{B} \mid Sa \mid aB \mid a$$

$$B \rightarrow bB \mid b$$

- Chomsky Normal form is obtained as:

$$S' \rightarrow ST \mid SA \mid A\mathbf{B} \mid a$$

$$S \rightarrow ST \mid SA \mid A\mathbf{B} \mid a$$

$$B \rightarrow CB \mid b$$

$$T \rightarrow CB$$

$$A \rightarrow a$$

$$C \rightarrow b$$

- Greibach Normal form is obtained as:

$$S' \rightarrow aBZT \mid aZT \mid aBT \\ \mid aT \mid aBZA \mid aZA \mid aBA \\ \mid aA \mid aB \mid a$$

$$S \rightarrow aBZ \mid aZ \mid aB \mid a$$

$$B \rightarrow bB \mid b$$

$$T \rightarrow bB$$

$$A \rightarrow a$$

$$C \rightarrow b$$

$$Z \rightarrow aBZ \mid aZ \mid aB \mid a$$

abaaba

G

CNF

GNF

$S \Rightarrow SaB$
 $\Rightarrow SaBaB$
 $\Rightarrow SaBaBaB$
 $\Rightarrow aBaBaBaB$
 $\Rightarrow abaBaBaBaB$
 $\Rightarrow abaBaBaB$
 $\Rightarrow abaaBaB$
 $\Rightarrow abaabBaB$
 $\Rightarrow abaabaB$
 $\Rightarrow abaaba$

$S \Rightarrow SA$
 $\Rightarrow STA$
 $\Rightarrow SATA$
 $\Rightarrow ABATA$
 $\Rightarrow aBATA$
 $\Rightarrow abATA$
 $\Rightarrow abaTA$
 $\Rightarrow abaABA$
 $\Rightarrow abaaBA$
 $\Rightarrow abaabA$
 $\Rightarrow abaaba$

$S \Rightarrow aBZA$
 $\Rightarrow abZA$
 $\Rightarrow abaZA$
 $\Rightarrow abaaBA$
 $\Rightarrow abaabA$
 $\Rightarrow abaaba$

Examples

- Convert to GNF
- $S \rightarrow aSX \mid b$
- $X \rightarrow Xb \mid a$